

Universidad Veracruzana

Facultad de Instrumentación Electrónica

Ingeniería Biomédica

EE: Interfaces Hombre-Máquina

Actividad 02: Botón de toque como dispositivo para una interfaz Hombre-Máquina

Profesor:
Varela Lara Luis Julian

Integrantes del equipo *"ES3.14"*:
Alfonso Gamboa Rubén
Flores Montero Edsel Yetlanezi

Fecha: 8 de abril de 2026

Xalapa, Enríquez, Ver.

I. Introducción al tema

Durante el desarrollo de esta práctica se pondrá en práctica lo aprendido en clase anteriormente: el uso de la función *milis* en lugar del uso de la función *delay*, pues la función *delay* detiene el programa una cantiad específica de milisegundos, con la desventaja que detiene todas las acciones durante su uso [1]. Mientras que *milis* no es una función de bloqueo [1]; ésta devuleve los milisegundos que han pasado desde que se ejecutó el programa, pudiendo usar este valor para calcular la diferencia de tiempo que hay entre un evento y otro. Función a la que se le sacó ventaja para el desarrollo de esta actividad.

II. Dearrollo de la actividad

II-A. Desarrollo del código

Para desarrollar el código primeramente se descomprime ComandosRS232.zip (descargado de la Actividad 01), obteniendo así nuestra primera dificultad: el fichero ComandosRS232.exe.

Para poder ejecutar ComandosRS232.exe se instala wine-11.4 (Staging). Fallando e inutilizando la GUI (figura 1), por lo que se opta por trabajar con arduino tui.

Durante el desarrollo del código no se contaba con el material, sin embargo se sabía de atemano que su comportamiento no debía ser muy diferente de lo de un botón pulsador [2], por lo que para las pruebas, se iban a realizar con botones pulsador.

Primeramente en el código se declaran funciones y variables que se van a ocupar posteriormente: BotonUp y BotonDown van a ser asignados a los pines 2 y 3 respectivamente,

Se crean variables con el prefijo *"unsidgned long"* , estas variables nos van a servir para guardar *tiempo*, por lo que se usa *long* para indicar que es un número muy grande, y como no vamos a trabajar con números negativos, se indica el prefijo *unsigned* (sin signo), pudiendo guardar números más grandes pero sin signo.

Se crea una función llamada *calcularIntervalo()*, que retornará un número entero (*int*) y que recibe de parametro tiempo (con las mismas condiciones que hemos usado para las variables del principio), esta función la queremos para decirle al Arduino la velocidad (en milisegundos) a la que queremos que incremente la variable **valor**.

Dentro del setup, le decimos a arduino que queremos ocupar 9600 baudios (estándar en arduino) y que queremos que los pines 2 y 3 serán inputs.

Dentro del loop leemos si los botones están presionados, guardamos esta información en variables booleanas, pues nos permiten usarlas como condicionales, facilitando la lectura del código. Además en esta parte añadimos una variable de control llamada *ahora*, que nos va a permitir saber *en que momento de milisegundo nos encontramos ahora mismo desde que se inició el programa.*

Primer condicional: Si ambos botones son presionados. Este primer condicional nos permite conocer si ambos botones se encuentran presionados, si esta oración es cierta, cambiamos el estado de amobos a *no presionados*, probocando que el programa no haga nada.

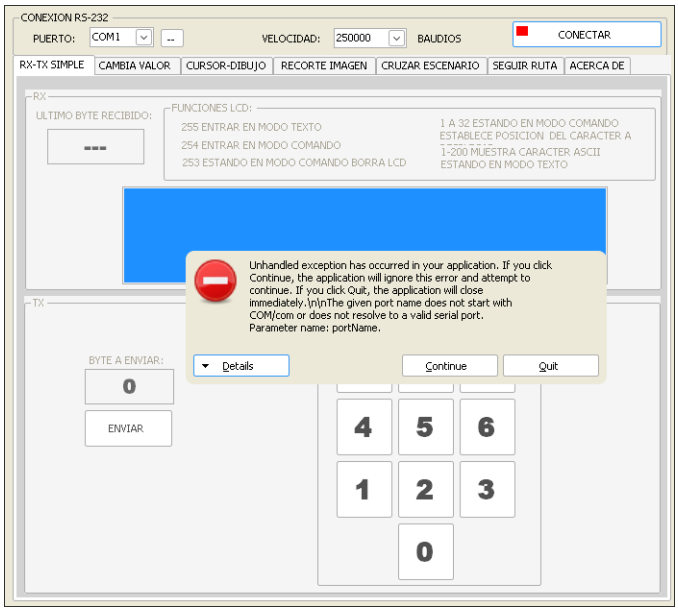


Figura 1. Falla del programa ComandosRS232

Segundo condicional: si UP es presionados.

1. Se anida un *if* que nos permite registrar en la variable de control *upPresionado* a *verdadero* si esta no lo estaba, guardando en el proceso, cuando fue presionado y el utlimo cambio.
2. Se calcula cuanto tiempo lleva presionado el botón: restando el tiempo de ahora, menos el momento en el que se presionó.
3. Con la función anterior de *calcularIntervalo()* se le asgina un intervalo de tiempo en el que tiene que imprimir el valor.
4. Finalmente, si la diferencia de los cambio, supera al intervalo (dado por el tiempo presionado), se imprime el valor. y se regisra el último cambio. Este último *if* es el causante del aumento de la velocidad, al evaluar constantemente *¿Debo imprimir ahora mismo?*

Finalmente para el **Tercer condicional** principal se repite la misma lógica que en el segundo, pero aplicado para el boton para bajar el valor.

II-B. Armado del circuito

Para el armado del circuto Se ocupó de un Arduino Uno, una protoboard, jumpers, los botones capacitivos fueron conectados respetando sus etiquetas VCC, GND y I/O:

1. Los pines de VCC fueron conectados en común al puerto de 5v del Arduino Uno.
2. Los pins GND fueron conectados en común al puerto GND del Arduino Uno.
3. El pin I/O del primer botón se conectó al puerto digital 2 del Arduino, este botón será para incrementar el valor.
4. El pin I/O del segundo botón se conectó al puerto digital 3 del Arduino, este botón será para decrementar el valor.

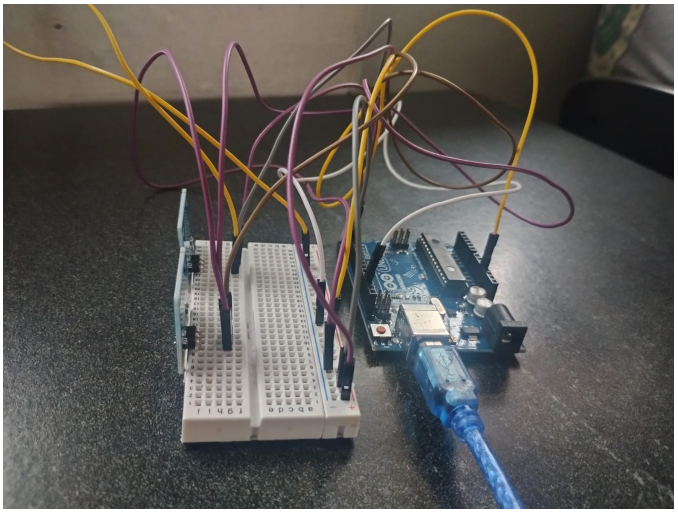


Figura 2. circuito armado

II-C. Código completo

A continuación se deja el código completo usado durante el desarrollo de esta actividad:

```
int botonUp = 2; //pin 2 para sumar
int botonDown = 3; // pin 3 para restar

int valor = 0;

// --- Variables para guardar tiempo, usamos unigned
↪  porque sologuardaremos númreos positivos
unsigned long tiempoInicioUp = 0;
unsigned long tiempoInicioDown = 0;

unsigned long ultimoCambioUp = 0;
unsigned long ultimoCambioDown = 0;

// Variables que funcionaran como banderas para
↪  indicar cuando se presiona un boton
bool upPresionado = false;
bool downPresionado = false;

// Función para calcular el tiempo entre un momento
↪  a otro (devuelve milisegundos a usar)
int calcularIntervalo(unsigned long tiempo) {

  if (tiempo < 500) return 400;
  if (tiempo < 2000) return 200;
  if (tiempo < 4000) return 100;

  return 50;
}

void setup() {

  Serial.begin(9600);

  pinMode(botonUp, INPUT);
  pinMode(botonDown, INPUT);
}

void loop() {

  bool up = digitalRead(botonUp);
  bool down = digitalRead(botonDown);

  unsigned long ahora = millis();

  // Si ambos se presionan no hacer nada
  if (up && down) {
    upPresionado = false;
    downPresionado = false;
    return;
  }

  // si se presiona el BOTON UP
  if (up) {

    if (!upPresionado) {
      tiempoInicioUp = ahora;
      ultimoCambioUp = ahora;
      upPresionado = true;
    }

    unsigned long tiempoPresionado = ahora -
    ↪  tiempoInicioUp;

    int intervalo =
    ↪  calcularIntervalo(tiempoPresionado);

    if (ahora - ultimoCambioUp ⩾ intervalo) {

      valor++;
      Serial.println(valor);

      ultimoCambioUp = ahora;
    }

  } else {

    upPresionado = false;
  }

  //si se presiona el BOTON DOWN
  if (down) {

    if (!downPresionado) {
      tiempoInicioDown = ahora;
      ultimoCambioDown = ahora;
      downPresionado = true;
    }

    unsigned long tiempoPresionado = ahora -
    ↪  tiempoInicioDown;

    int intervalo =
    ↪  calcularIntervalo(tiempoPresionado);

    if (ahora - ultimoCambioDown ⩾ intervalo) {

      valor--;
      Serial.println(valor);

      ultimoCambioDown = ahora;
    }

  } else {
```

```
    downPresionado = false;
  }
}
```

III. Conclusiones

La mayor dificultad para el desarrollo de esta práctica fue el mal funcionamiento del **ComandosRS232.exe** dentro del entorno linux, lo que llevó a una malinterpretación de funciones y objetivos por parte del equipo, por lo que se optará por el uso de maquinas virtuales para el desarrollo de las futuras actividades.

El uso de la función *milis*, permite un uso continuo del microcontrolador Arduino Uno sin necesidad de hacer pausas que congelen al microcontrolador. Permitiendo un uso continuo de éste para otras tareas. El uso de botones touch son una alternativa moderna e ingeniosa a los botones que se pulsan, evitándonos los problemas comunes del polvo y el desgaste del resorte.

IV. firma de revisión

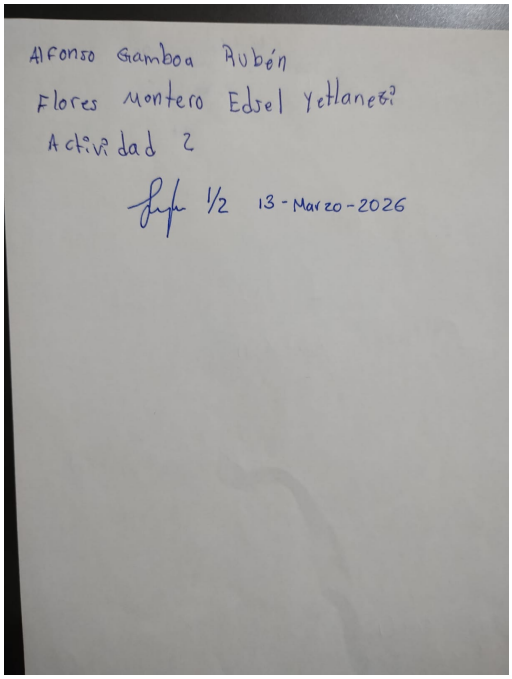


Figura 3. Firma de revisión

Referencias

[1] Industrial Shields, *Industrial Arduino Millis() vs Delay()*, https://www.industrialshields.com/es_ES/blog/blog-industrial-open-source-1/industrial-arduino-millis-vs-delay-248, Accedido: 2026-03-13, sep. de 2020.

[2] R. Keim, *Introduction to Capacitive Touch Sensing*, https://www.allaboutcircuits.com/technical-articles/introduction-to-capacitive-touch-sensing/, Uyaykusqa: 2026-03-13, mayo de 2016.